

Autonomous Drone Navigation Using Deep Reinforcement Learning

COMP9444 Project Team

Abstract—This project develops a visual deep reinforcement learning system for autonomous multirotor navigation in Microsoft AirSim. The drone observes an 84 x 84 front depth image together with relative goal position and velocity, and chooses among six discrete body-frame actions without an obstacle map. We implement a vanilla Deep Q-Network (DQN) baseline, a stabilised Proximal Policy Optimisation (PPO) agent trained directly on the final route, and PPO trained through a three-stage distance curriculum. All methods consume 45,000 environment interactions and are selected by validation before an independent 50-episode test. Deterministic PPO trained from scratch achieves 98% success with 2% collision, compared with 16%/72% for DQN and 68%/2% for curriculum PPO. The results show that stable on-policy optimisation can learn a repeatable depth-driven detour, while the curriculum reduces collisions but introduces a 30% timeout rate. The evidence is limited to one route and one training seed, so cross-route generalisation remains future work.

Keywords—AirSim, autonomous drone navigation, deep reinforcement learning, DQN, PPO, curriculum learning.

I. INTRODUCTION

Autonomous flight requires a controller to convert high-dimensional visual observations into safe actions under uncertain geometry. Hand-engineered planners usually depend on maps, calibrated sensing, or explicit obstacle models; these assumptions are difficult to maintain in cluttered and changing environments. Deep reinforcement learning (DRL) offers an alternative in which perception and action selection are learned jointly from simulator interaction.

We study point-to-point navigation in the AirSimNH neighbourhood scene. The agent must travel approximately 32.55 m, avoid houses and vegetation, remain within a safe altitude band, and stop within a 2 m goal radius. The central question is whether PPO learns this visual route more reliably than a vanilla DQN baseline, and whether a progressive target-distance curriculum improves PPO under the same interaction budget.

Our work is the AirSim-to-Gymnasium environment, visual/state preprocessing, common CNN policy interface, reward and safety design, stable PPO modifications, three-stage transfer procedure, per-step diagnostics, checkpoint selection, and controlled evaluation. AirSim, DQN, PPO, and the curriculum principle originate from prior work [1]–[4]; the implementation and experiments reported here are our own.

II. RELATED WORK

AirSim provides an Unreal Engine-based visual and physical simulator for autonomous vehicles, including multirotor dynamics, cameras, collision queries, and programmatic control [1], [5]. Its repeatability and realistic sensing make it suitable for training without risking physical hardware, although simulator performance alone does not establish real-world transfer.

DQN demonstrated that convolutional networks can learn value functions directly from image observations in discrete-action tasks [2]. Its replay buffer and target network improve stability, but off-policy value learning can be sensitive to sparse terminal rewards and correlated failure modes. PPO instead updates a stochastic actor-critic using a clipped surrogate objective, permitting multiple minibatch epochs while limiting destructive policy changes [3]. Curriculum learning orders training tasks from easier to harder [4]; here, target distance is increased across stages. Unlike these foundational benchmarks, our comparison holds the visual encoder, action space, reward, final route, and total interactions constant.

III. RL TASK AND METHODS

A. Observation, Actions, and Reward

There is no offline dataset or class label. AirSim generates one transition per interaction. The visual input is a front DepthPerspective frame clipped to 40 m, converted to obstacle intensity 1-depth/40, resized to 84 x 84, and stored as a single normalised channel. A six-dimensional state appends scaled relative goal position ($dx/50$, $dy/50$, $dz/10$) and velocity ($vx/10$, $vy/10$, $vz/10$). The policy receives neither a map nor obstacle coordinates; collision state is used only for reward, termination, and logging.

The discrete actions are forward, left, right, up, down, and hover. Each command lasts 0.35 s at 2 m/s horizontally or 1 m/s vertically. The shaped reward is $r = -0.05 + 2(d_{\text{prev}} - d) - 0.25|z - z_{\text{target}}|$ plus a boundary-margin term. Reaching the goal adds +100; collision or

altitude violation adds -100; a 150-step timeout adds -25. Episodes terminate on success, a new collision, altitude outside $-10 \leq z \leq -1$ m in NED coordinates, or timeout.

B. Shared Visual Encoder and DQN

All agents use three ReLU convolutional layers: 32 filters (8 x 8, stride 4), 64 filters (4 x 4, stride 2), and 64 filters (3 x 3, stride 1). The resulting 3,136 visual features are concatenated with the six navigation features. DQN maps the 3,142-vector through a 512-unit ReLU layer to six Q-values. It uses $\gamma=0.99$, Adam at $1e-4$, batch size 32, a 50,000-transition uniform replay buffer, Huber TD loss, gradient clipping, a target update every 1,000 interactions, and epsilon decay from 1.0 to 0.05 over 50,000 steps. It intentionally omits Double, Dueling, prioritised replay, and n-step extensions, so it is a transparent baseline rather than an optimised DQN.

C. Stable PPO and Curriculum

PPO feeds the shared representation through a 512-unit layer, per-sample layer normalisation, and Tanh. Separate linear actor and critic heads output six logits and one state value. Orthogonal initialisation, reward scaling by 0.1, Huber value loss, advantage normalisation, gradient clipping, and a linear entropy schedule from 0.01 to 0.001 address the activation saturation observed in an earlier implementation. Training uses $\gamma=0.99$, GAE $\lambda=0.95$, clip coefficient 0.2, 500-step rollouts, batch size 64, four epochs, and Adam at $1e-4$.

Scratch PPO receives all 45,000 interactions on the final 33 m task. Curriculum PPO uses 5,000 interactions to a 10 m target, 10,000 to a 23 m target, and 30,000 to the final target; maximum episode lengths are 70, 110, and 150. The selected policy weights transfer between stages while Adam state is reset; Stage 3 uses a $7.5e-5$ learning rate. This equalises total interactions but not final-task exposure, which is central to interpreting the result.

Fig. 1. Training success is highly variable. Curriculum stages before 15k interactions use easier targets; only Stage 3 matches the final route.

IV. EXPERIMENTAL SETUP

Experiments use AirSimNH with start (85.413, -15.334, -3.0) and final target (117.756, -19.034, -3.0). All three methods use Seed 7, 45,000 new environment interactions, the same observation/action/reward definitions, and a 150-step final-task limit. Measured training times were 7.41 h for DQN, 6.50 h for Scratch PPO, and 6.51 h for all curriculum stages combined.

To reduce checkpoint-selection bias, every checkpoint is first evaluated for five episodes with the training seed; the top three are then evaluated for 30 episodes with Seed 10007. The selected model is tested once for 50 fresh episodes with Seed 20007. This seed changes stochastic simulator and policy sampling but not the route geometry. The primary deployment metric is deterministic success; secondary metrics are collision, timeout, altitude violation, reward, steps, path length, final distance, and minimum perceived depth. PPO is additionally evaluated stochastically to diagnose the learned action distribution.

V. V. RESULTS

TABLE I. INDEPENDENT FINAL TEST (DETERMINISTIC, N=50)

Method	Succ.	Coll.	TO	Reward	Dist.
DQN-S	16%	72%	14%	-18.5	4.47
PPO-S	98%	2%	0%	141.1	2.01
PPO-C	68%	2%	30%	71.2	14.66

Succ./Coll./TO are success, collision, and timeout rates; Dist. is mean final distance in metres.

Success Collision Timeout

Fig. 2. Independent deterministic outcomes. Rates are reported separately because a terminal event at the step limit can overlap timeout bookkeeping.

Scratch PPO succeeds in 49/50 episodes (98%; Wilson 95% CI 89.5-99.6), versus 8/50 for DQN (16%; 8.3-28.5) and 34/50 for curriculum PPO (68%; 54.2-79.2). It also obtains the highest reward (141.1), shortest mean completion time (52.28 steps), and a 1.92 m mean minimum depth. Its 38.27 m path is 17.6% longer than the straight-line distance, consistent with a deliberate lateral detour rather than direct pursuit.

DQN collides in 72% of episodes, reaches a minimum perceived depth of only 0.37 m on average, and often alternates direct motion with hover near the obstacle. Curriculum PPO has the same low 2% deterministic collision rate as Scratch PPO, but times out in 30% of episodes and therefore ends 14.66 m from the target on average. Its 55.17 m average path indicates inefficient wandering in failed episodes.

AirSim NED x (m)

Fig. 3. Representative deterministic Episode 1 trajectories. Both PPO policies move laterally before approaching the target; DQN follows a lower-clearance route and collides.

The deployment mode matters. Scratch PPO falls from 98% deterministic success to 66% when actions are sampled; curriculum PPO falls from 68% to 46% and its collision rate rises from 2% to 38%. Thus the argmax policy is repeatable, but probability mass remains on unsafe alternatives. The reported mean final distance of 2.01 m for Scratch PPO is compatible with a 2 m success radius because it averages 49 near-boundary successes with one collision failure.

VI. VI. DISCUSSION AND CONCLUSION

The controlled comparison supports three conclusions. First, PPO is substantially better matched to this visual navigation task than the implemented vanilla DQN: clipped on-policy updates, GAE, and actor-critic learning produce a stable detour instead of repeatedly bootstrapping values from collision-heavy experience. Second, curriculum learning is not automatically beneficial. Spending one third of the budget on shorter goals reduces final-route exposure, and changing targets creates a distribution shift; the transferred policy is safe when successful but preserves short-horizon behaviour that manifests as timeouts. Third, deterministic and stochastic evaluation answer different questions: argmax measures deployment performance, while sampling reveals residual uncertainty.

Strengths of the system include map-free visual control, a fair interaction budget, identical final-task evaluation, explicit safety metrics, checkpoint selection separated from testing, and action/reward trajectory logs that make failure analysis reproducible. The main weakness is external validity. There is only one training seed, one fixed route, one scene, one front depth frame, and no recurrent memory, yaw, backward action, domain randomisation, or real-drone validation. The test seed is independent of checkpoint selection but does not create unseen geometry, so these results must not be described as generalisation.

Future work should repeat training across at least three to five seeds, perturb safe start and target positions, evaluate a second AirSim environment, and train on multiple routes with weather and texture randomisation. Algorithmic extensions should compare Double/Dueling DQN, recurrent visual memory, continuous-control PPO or SAC, and a curriculum that preserves final-task rehearsal. Within the current scope, deterministic Scratch PPO is the

recommended policy: it achieves 98% success and 2% collision on the held-out 50-episode fixed-route test.

VII. REFERENCES

- [1] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," in *Field and Service Robotics*, 2018, pp. 621-635.
- [2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529-533, 2015.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- [4] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in *Proc. ICML*, 2009, pp. 41-48.
- [5] Microsoft Research, "AirSim," GitHub repository. Available: <https://github.com/microsoft/AirSim>.